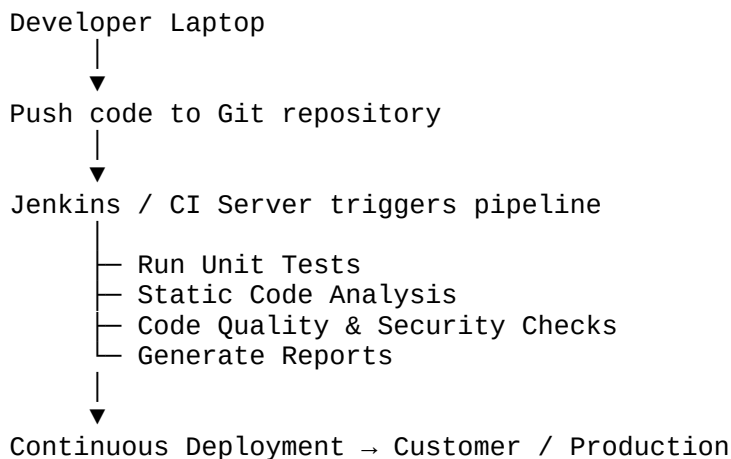


CI/CD (Continuous Integration / Continuous Delivery) Overview

Stage	Description	Tools / Examples
1. Continuous Integration (CI)	Developers frequently integrate code into a shared repository (e.g., GitHub). Jenkins or other CI tools detect changes automatically.	Git, Jenkins, GitLab CI, Travis CI
2. Personal Laptop → Customer / Shared Environment	Developers work locally, push changes to a shared repo where CI/CD pipeline runs.	Git, Jenkins
3. Unit Testing	Automated tests run to check individual functions or modules work correctly.	JUnit, pytest, NUnit
4. Static Code Analysis	Checks source code for errors, code style, security vulnerabilities before runtime.	SonarQube, ESLint, Pylint
5. Code Quality / Vulnerability Check	Ensures maintainability, security, and standards compliance.	SonarQube, Fortify, Checkmarx
6. Automation	Automates build, test, and deployment steps to reduce manual errors.	Jenkins, Maven, Gradle, Docker
7. Reports / Notifications	Generates test reports, code quality dashboards, and sends notifications to the team.	Jenkins, Slack, Email, HTML reports
8. Deployment (CD)	Automatically deploys the tested and approved code to staging or production environments.	Jenkins, Docker, Kubernetes, Ansible

1 CI/CD Flow (High-Level)



Key Points:

- **CI** focuses on **integration and testing** frequently.
- **CD** focuses on **delivery and deployment** to production or staging.
- Automation ensures faster, reliable, and consistent releases.

What Jenkins Does

- Jenkins is a **CI/CD automation server**.
 - It **watches your Git repositories** (GitHub, GitLab, etc.) for changes:
 - New commits
 - Pull requests
 - Branch updates
 - When Jenkins detects a change, it **triggers a pipeline**, which is a series of automated steps such as:
 - Building the code
 - Running **unit tests**
 - Performing **static code analysis**
 - Generating reports
 - Deploying to staging or production
 - Jenkins can also **notify you** about:
 - Build successes or failures
 - Test results
 - Code quality issues
-

Simplified Flow

Code pushed to GitHub



Jenkins detects change



Pipeline runs automatically



Build Test Deploy



Notifications sent to developers

Key idea: Jenkins automates the process so developers don't have to manually build, test, or deploy every time there's a change.

Jenkins can promote an application to **different environments** automatically as part of a CI/CD pipeline. Common promotion stages include:

Stage / Environment	Purpose
Development (Dev)	Initial testing by developers; run unit tests and code quality checks.
Integration / QA	Combined testing with other modules; run functional and integration tests.
Staging / Pre-Prod	Almost production-like environment; final validation before release.
Production (Prod)	Live deployment for end-users.

Jenkins pipelines can automate **build** → **test** → **deploy** → **notify** across these stages.

What is an EC2 Instance?

EC2 stands for **Elastic Compute Cloud**. It is a service provided by **AWS (Amazon Web Services)** that lets you run virtual computers in the cloud.

Think of it like **renting a computer** on the internet. You don't have to buy a physical machine; AWS provides a virtual machine with CPU, memory, storage, and networking.

Key points about EC2:

1. Virtual Server:

- An EC2 instance is essentially a virtual server running on AWS's hardware.
- You can choose different sizes (CPU/RAM/storage) depending on what you need.

2. Operating System:

- You can select the OS you want: Ubuntu, CentOS, Windows, etc.

3. Access:

- You connect to it via **SSH** (for Linux) or **RDP** (for Windows).
- You can install software, run applications, and configure it just like a normal computer.

4. Elasticity:

- You can start and stop instances anytime.
- You can scale up (more resources) or scale down depending on demand.

5. Security:

- EC2 instances have **Security Groups**, which act like firewalls to control who can access your instance.
- Example: If Jenkins runs on port 8080, you need to allow traffic on that port to access it from your browser.

Where you install	Effect on physical laptop?	Always on?
EC2 instance	No effect at all	Yes, as long as EC2 is running
Friend's laptop	Only affects that laptop	Only if laptop is on

1 Imagine a real project

- You and your team are building a **website**.
 - The code is stored in **GitHub**.
 - Developers keep **adding new features or fixing bugs**.
-

The old way (manual)

1. Developer pushes code to GitHub.
2. Another person manually downloads the code.
3. They compile/build the application.
4. They test it manually.
5. They upload it to the server for users.

Problem: This takes time, mistakes happen, and it slows down the team.

The modern way with Jenkins on EC2

1. **Set up Jenkins** on an **EC2 instance** (a virtual server always online).
 2. **Connect Jenkins to GitHub:** Jenkins watches your repository.
 3. **Automatic actions (CI/CD pipeline):** Every time a developer pushes code:
 - Jenkins automatically **pulls the new code**.
 - Jenkins **builds the application** (turns code into a working program).
 - Jenkins **runs automated tests** to check if the new code broke anything.
 - Jenkins **deploys the application** to another server (EC2 or cloud service) where users can see it.
-

Benefits

- No one has to do these steps manually.
- Builds and tests happen **instantly** whenever code changes.
- Fewer human errors.
- Developers can focus on coding instead of deployment.

Jenkins

Jenkins watches GitHub, pulls the code, runs the pipeline to build/test/package the application, and optionally deploys it or creates artifacts.

Jenkins is an open-source automation server that continuously monitors source code repositories (e.g., GitHub), automatically pulls changes, executes defined pipelines for building, testing, and packaging applications, and can optionally deploy the artifacts to target environments, enabling automated Continuous Integration (CI) and Continuous Delivery (CD) workflows.

1 Key Points About Jenkins

1. What it is:

- Jenkins is an **open-source automation server** used for **CI/CD (Continuous Integration / Continuous Deployment)**.

2. Purpose:

- Automates building, testing, and deploying applications.
- Reduces manual work and human errors in software delivery.

3. How it works:

- Developers push code to a **repository** (GitHub, GitLab, etc.).
- Jenkins **pulls the code**, builds it, runs tests, and deploys it automatically.

4. Runs on:

- Jenkins can run on **any server**: local machine, company server, or **cloud instance (like AWS EC2)**.

5. Architecture:

- **Master**: Controls the jobs, schedules builds, monitors agents.
- **Agents/Workers**: Machines (physical or virtual) that execute the build tasks.

6. Plugins:

- Jenkins has **thousands of plugins** to integrate with tools like Git, Docker, Kubernetes, Slack, etc.

7. Web Interface:

- Jenkins provides a **web dashboard** for easy job creation, monitoring builds, and viewing logs.

8. Supports multiple environments:

- Can build and deploy applications for **different operating systems and platforms**.

9. Pipeline as Code:

- Jenkins supports **pipeline scripts** (Jenkinsfile) to define build and deployment steps as code.

10.Triggering builds:

- Builds can be triggered **manually**, **on code push**, or **on a schedule** (cron jobs).

11.Security:

- Supports **user authentication**, role-based access, and **credentials management**.

12.Use in real projects:

- Used by companies to **automate testing, deployment, and delivery** of software.
- Example: Developers push code → Jenkins builds → Jenkins runs tests → Jenkins deploys to production automatically.

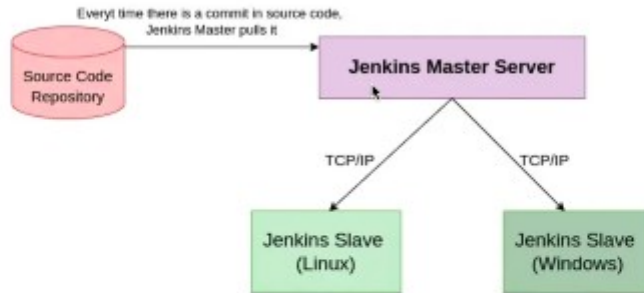
Example: Installing Jenkins on AWS EC2

- Refer readme of this github repo : <https://github.com/iam-veeramalla/Jenkins-Zero-To-Hero/>
- Start with an **EC2 instance** in AWS (Amazon Web Services)
- Copy the **instance public IP address**
- **SSH** to that EC2 instance using your **key-pair**
- Jenkins is a **Java-based application**, so you need to have **Java as a prerequisite**
- Run `sudo apt update`
- **Install JDK**
- **Verify Java installation** using `java -version`
- **Install Jenkins**
- By default, **EC2 instance will not accept traffic from external world**, inbound traffic rules are blocked..so if u want to access any application like jenkins.
- Jenkins is an application running on **port 8080**
- Check if Jenkins is running using `ps -ef | grep jenkins`
- AWS is blocking port 8080, so you need to allow traffic
- Go to **Security** → **Existing Security Groups** → **Edit inbound rules** → **Add rule**
 - Type: **All Traffic**
 - Port range: **8080**

- Source: **Anywhere**
- Save rules
- Now got to ec2 instance, copy public ip address
- Open a browser and enter `http://<EC2-Public-IP>:8080`
- This allows you to **access Jenkins running on the EC2 instance**
- Jenkins will prompt for the **initial admin password** (stored in `/var/lib/jenkins/secrets/initialAdminPassword`)
- `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
- **Copy this password** and paste it into the Jenkins browser page to unlock Jenkins
- You are now able to **access the Jenkins dashboard**
- Jenkins gives you **two options**:
 - **Install suggested plugins**
 - **Select plugins to install**
- Go with **Install suggested plugins**
- Jenkins will **show a list of plugins** being installed
- At any point, you can **install additional plugins** later from the dashboard
- If plugin installation is taking time, you can choose **Install selected plugins** instead
- **Create the first admin user**: enter **username, password, full name, and email**
- Configure **instance settings**: Jenkins URL (IP address and port)
- Once done, you can **start using Jenkins**
- **Jenkins installation part is now complete**

Jenkins Architecture

Working of Jenkins Architecture



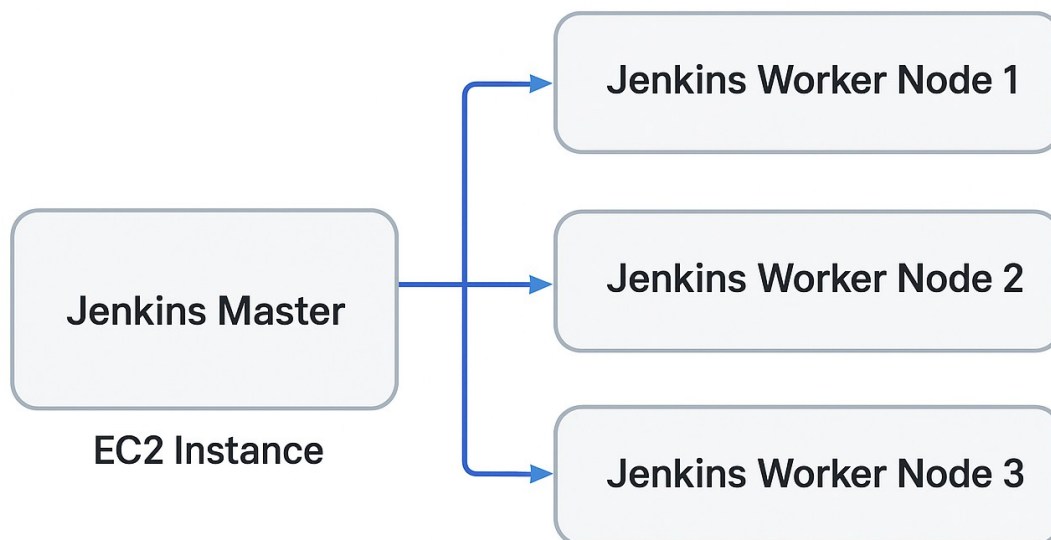
SOURCE CODE REPOSITORY-

A REPOSITORY THAT CONTAINS THE SOURCE CODE AND ALL OTHER REQUIRED PROJECT-RELATED FILES.

MASTER--

THE MAIN JENKINS SERVER IS THE MASTER SERVER OF JENKINS. THIS ACTS AS A CONTROL SERVER THAT ARRANGES THE DEFINED WORKFLOW OF THE PIPELINE. THE JOBS OF THIS MASTER ARE AS FOLLOWS:

- SCHEDULING JOBS
- SENDING BUILDS TO SLAVES TO EXECUTE THE JOB
- KEEPING TRACK OF THE SLAVES STATES (ONLINE/OFFLINE)
- CAN ALSO DIRECTLY EXECUTE BUILD JOBS



- **Jenkins Master** (on the left side in diagrams)
 - This is where **Jenkins is installed** (in your example, the EC2 instance acts as the Master)
 - Responsible for **scheduling jobs, managing the build queue, and monitoring workers**

- Should **not do heavy builds** if multiple developers are pushing code, to avoid overload
- **Jenkins Worker Nodes / Agents** (usually on the right side)
 - Worker nodes **perform the actual build, test, and deployment tasks**
 - Can have **multiple nodes**: Node1, Node2, Node3, etc.
 - Helps **distribute the workload** and speed up CI/CD
- **Communication:**
 - Master sends tasks to worker nodes
 - Worker nodes execute tasks and send results back to the master
- **Load Management:**
 - Master is mainly for **orchestration**
 - Worker nodes handle **heavy lifting** like compiling, testing, or deploying

1 Jenkins Master:

- The EC2 instance you currently have is running **Jenkins Master**.
- Its primary role is **orchestration**: scheduling builds, managing worker nodes, triggering jobs, and storing configuration.
- **Important:** Jenkins Master should **not** run all builds directly because:
 - Builds require different environments (Java 7 vs 8, Python 2 vs 3, etc.).
 - Running everything on Master would overload it.
 - Dependency conflicts between projects could occur.

2 Jenkins Worker Nodes (Agents):

- These are additional EC2 instances (or VMs/containers) connected to the Master.
- They **actually run the build tasks**, tests, or deployments.
- Worker nodes can be specialized:
 - By application type (Windows vs Linux).
 - By project team.
 - By technology stack (Python, Java, Node.js, etc.).
- This helps avoid idle Master time, dependency conflicts, and resource overload.

3 Load Distribution:

- If you have 1 Master and 20 Worker nodes, the Master just schedules jobs.
- Workers pick up jobs according to labels, resource availability, or team assignment.
- This allows scaling in microservices environments where builds and deployments are frequent.

4 Modern Context (Kubernetes / Microservices):

- With containerized microservices, the static worker-node model can leave nodes idle if no jobs target them.
- Dynamic provisioning (like Kubernetes Jenkins agents) can spin up agents **on-demand** and tear them down after use.
- This optimizes resource usage, unlike the older static EC2-based worker node setup.

Problem with Idle Worker Nodes:

- **You had Jenkins Master on one EC2.**
- Each worker node was a separate EC2 instance.
- Problem: idle nodes waste money; environment setup is rigid.

Main Issues

1. Idle Nodes Waste Money

- If no jobs are scheduled for a worker node (e.g., “Worker Node 3”), it **sits idle**, still costing money because AWS charges for uptime.

2. Environment Setup is Rigid

- Each EC2 node must be **pre-configured** for a specific environment:
 - Windows or Linux
 - Python 3.x, Node.js 16, Java 17, etc.
- **Problem:**
 - You cannot predict which environment the next job will need.
 - To avoid delays, organizations may keep multiple EC2 nodes “just in case,” which wastes money.

3. Auto-scaling Helps But Has Limitations

- You still need **at least one node always available** to handle incoming jobs immediately.
 - You **don’t know in advance** which environment the job requires.
 - Managing multiple EC2 images for every environment becomes cumbersome:
 - Example EC2 images:
 - Python 3.11
 - Node.js 16
 - Java 17
 - **Problems with multiple images:**
 - Updating or patching images takes time.
 - Maintaining consistency between images is hard.
 - Scaling quickly becomes messy when more workers are needed.
-

Summary

Problem Area	Issue
Idle Nodes	Costly because EC2 runs even when no jobs are assigned
Environment Setup	Each EC2 needs pre-configured software; hard to predict job needs
Auto-Scaling Limitations	Still need at least one node; managing multiple EC2 images is messy
Maintenance	Updating, patching, and keeping consistency across multiple images
Scalability	Difficult to scale quickly without creating more EC2s

1 Solution: Jenkins + Docker Agents:

- Instead of static EC2 nodes, you use **Docker containers** as ephemeral worker nodes.
- When a Jenkins job starts, it **spins up a container** with the exact environment it needs.
- After the job finishes, the container is destroyed.
- Benefits:
 - **Lightweight:** Containers are much faster and lighter than full VMs.
 - **Flexible:** Can create any environment on-demand (e.g., Node.js 16, Python 3.11, Java 17).
 - **Cost-efficient:** No idle EC2 resources; containers exist only for the job duration.
 - **Consistency:** The same container image ensures the environment is identical every time, avoiding “it works on my machine” issues.

2. Use Case Example:

- Today someone needs **Node.js 15**, tomorrow **Node.js 16**.
- With Docker, you just launch a container with the desired Node.js version; no need to pre-configure separate worker nodes.

3. Modern Jenkins Pipelines:

- Pipelines can specify a `agent { docker { image 'node:16' } }` for a stage.
- Jenkins dynamically provisions the container, runs the stage, then cleans up automatically.

In short: Docker + Jenkins agents = dynamic, on-demand worker nodes, solving idle resource problems and dependency conflicts.

2 Architecture

Jenkins Master —> EC2 Node1 (Python3)

EC2 Node2 (Java8)

EC2 Node3 (Node.js)

Problems:

- Idle EC2s waste money
- Environment conflicts (different software per node)
- Expensive to scale
- Hard to maintain multiple images

Jenkins Master —> Docker Container1 (Python3)

Docker Container2 (Java8)

Docker Container3 (Node.js)

Benefits:

- Containers spin up only when needed
- Any environment can be launched dynamically
- Lightweight, flexible, cost-efficient
- Clean and isolated environments per job

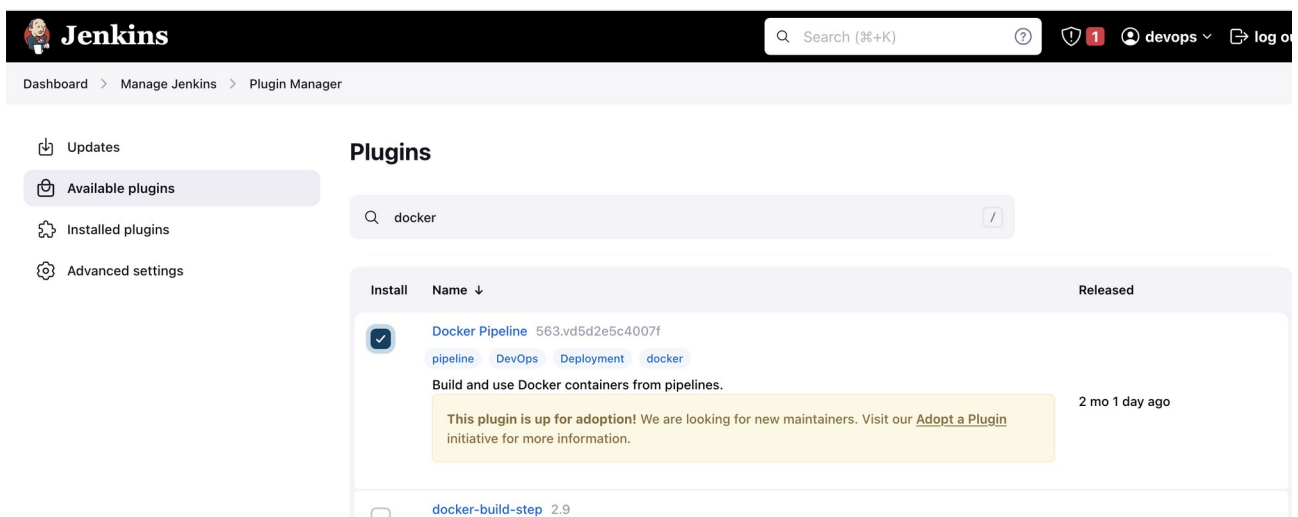
Docker

- Refer to <https://github.com/iam-veeramalla/Jenkins-Zero-To-Hero/> README
- `sudo apt update`
- `sudo apt install docker.io`
- `sudo su -`
- `usermod -aG docker jenkins`
- `usermod -aG docker ubuntu`
- `systemctl restart docker`
- Restart Jenkins: `http://<ec2-instance-public-ip>:8080/restart`
- If installing Docker on the same machine, it runs on Master
- If installing Docker on another machine, configure it as a Jenkins slave
- Docker runs as a daemon process, by default only accessible to root
- Grant access to Docker daemon for Jenkins and Ubuntu users
- Switch to root: `sudo su -`
- Add Jenkins and Ubuntu users to Docker group: `usermod -aG docker jenkins` and `usermod -aG docker ubuntu`
- Switch to Jenkins user: `su - jenkins`
- Test Docker: `docker run hello-world` to ensure Docker is installed and accessible
- If problem occurs, logout and re-add Jenkins to Docker group: `usermod -aG docker jenkins`
- Switch again to Jenkins: `su - jenkins`
- Run test again: `docker run hello-world`
- Jenkins user can now create and run containers
- Restart Docker daemon if needed: `systemctl restart docker`
- Jenkins user is now able to create and run Docker containers, which meets the requirement
- Ubuntu user was also granted Docker access to avoid potential issues when using Jenkins with the Ubuntu user
- Sometimes Jenkins does not immediately pick up these changes
- It is recommended to **restart Jenkins** after granting Docker permissions
- To restart Jenkins: go to `http://<jenkins-url>/restart`, confirm the pop-up, and Jenkins will restart

- Restarting ensures Jenkins picks up new permissions and avoids caching issues, especially if Jenkins has been running for a long time or new software has been installed
- login to jenkins
- done with installing docker

Install the Docker Pipeline plugin in Jenkins:

- Log in to Jenkins.
- Go to Manage Jenkins > Manage Plugins.
- In the Available tab, search for "Docker Pipeline".
- Select the plugin and click the Install button.
- Restart Jenkins after the plugin is installed.



- Either u can select install without restart or download now & install after restart. anything is fine
- press download now and restart later
- download progress
 - restart jenkins
 - login

Creating Jenkins Projects and Pipelines:

- To create your first Jenkins pipeline, click on **"New Item"**
- Jenkins offers multiple project types:
 - **Freestyle Project**
 - **Pipeline Project**
 - **Multi-configuration Project**
 - **Multibranch Pipeline**
 - **Organization Folder**
- Freestyle projects were the original project type available 5–10 years ago
- In a freestyle project, all configurations are provided through **input boxes** in the UI
 - Example: Git repository URL, credentials, build commands, etc.
- Advantages of freestyle projects:
 - Easy to use
 - No coding required
 - Configurations can be done via checkboxes and text fields
- Drawbacks of freestyle projects:
 - Not **declarative** → cannot be easily shared with team members
 - Cannot follow standard code review workflows (e.g., pull requests, peer reviews)
 - Hard to version control or replicate across environments
- To overcome these drawbacks, **Jenkins introduced Pipelines**, which are declarative and can be versioned in source control
- To overcome these issues, Jenkins introduced **Pipeline projects**
- Create your first Jenkins job:
 - Enter item name: `first-jenkins-job`
 - Select **Pipeline**
 - Choose between **declarative** or **scripted** pipeline
 - Write the pipeline as **code**
 - Define your own **stages and steps**
 - Store the pipeline in a **Git repository**
 - Easily modify the pipeline whenever new plugins are added

"Using the First Jenkins Pipeline from GitHub"

- Repository: [Jenkins-Zero-To-Hero / my-first-pipeline](#)
- Start with the **basic pipeline** from the GitHub repository
- The folder contains **three Jenkins projects**, the first being “**my first pipeline**”
- Purpose of the first pipeline: verify if the **Docker agent configuration** is working correctly

```
pipeline {
  agent {
    docker { image 'node:16-alpine' }
  }
  stages {
    stage('Test') {
      steps {
        sh 'node --version'
      }
    }
  }
}
```

This pipeline uses a Docker container as the agent, runs a single “Test” stage, and executes a shell command to check the Node.js version. It demonstrates how to run a Jenkins pipeline in a reproducible environment using Docker.

1 pipeline { ... }

- Defines a **Jenkins Pipeline** block. Everything inside this block is part of the pipeline definition.

2 agent { docker { image 'node:16-alpine' } }

- Tells Jenkins to run this pipeline inside a **Docker container**.
- `image 'node:16-alpine'` specifies the **Docker image** to use (node:16-alpine is a lightweight Node.js 16 image).
- This ensures the pipeline runs in a clean environment with Node.js 16 pre-installed.
- • Today you are using Node.js 14, tomorrow you want Node.js 15; instead of manually logging into virtual machines to update Node.js, dependencies, and handle security or package.json changes, you simply update the Docker image in the Jenkinsfile, and all pipeline runs automatically use the new environment. this is the advantage of using docker as agent

-

3 **stages { ... }**

- Contains all the **stages** of the pipeline. Each stage represents a logical step in your CI/CD workflow.
- `stage('Test') { ... }`
- Defines a stage named “**Test**”.
- You can add multiple stages like Build, Test, Deploy, etc.

4 **steps { ... }**

- Contains the **actual commands** to execute within that stage.
- `sh 'node --version'`
- Runs a **shell command** inside the Docker container.
- Here it prints the installed Node.js version to verify that the Docker agent is working correctly
- Tomorrow, more steps will be added to this pipeline to extend its functionality

"Using Jenkins Pipeline Syntax Generator for Writing Pipelines"

- You can start learning Jenkins pipelines from a simple Hello World example or from the GitHub repository
- Jenkins provides a **Pipeline Syntax** utility to help generate Groovy code
- Example use case: checking out code from GitHub
 - Go to **Pipeline Syntax** in Jenkins
 - Select the option you want (e.g., Git)
 - Provide repository URL and branch name (Master/Main)
 - Optionally, select credentials if required
 - Click “**Generate Pipeline Script**” to get the ready-to-use Groovy code
 - u can copy this and put it in pipeline
- Running shell scripts in a pipeline:
 - Search for “**shell script**” in the Pipeline Syntax generator
 - Specify commands or script path (e.g., `cd Foo && ./hacktest.sh`)

- Jenkins generates the correct Groovy syntax to include in your pipeline
- Click “**Generate Pipeline Script**”
- **Jenkins Groovy Code Generator** is a very useful tool, especially for beginners, to learn and write pipelines
- Once you have the **Jenkinsfile**, you can directly use it in your pipeline without writing code manually
- This utility allows you to explore and execute **complex pipeline steps** with minimal errors and learning curve

First Jenkins Pipeline from SCM with Docker Agent

- **Open your first Jenkins job**
- Select “**Pipeline script from SCM**” instead of writing the pipeline manually
- Choose your source code repository (e.g., GitHub)
- Enter the repository URL
- Select the branch (Master/Main)
- Specify the path to the Jenkinsfile in the repository (e.g., `my-first-pipeline/Jenkinsfile`)
- Save the configuration

1 Pipeline Execution Flow:

- Click **Build Now** to trigger the pipeline
- Jenkins fetches the code from GitHub
- Jenkins checks for the specified Docker image (`node:16-alpine`)
 - If the image is not found, Docker pulls it automatically
- The pipeline executes inside the Docker container
 - In this example, it runs `node --version` to verify Node.js installation
- This demonstrates that the Docker agent configuration and pipeline setup are working correctly
- Even with only a single EC2 instance (Jenkins Master), the pipeline can run in an **isolated Docker environment** without needing separate worker nodes

2 Ephemeral Docker Containers (Run-on-Demand):

- A container is created **only when there is a request**
 1. “Request” means triggering the pipeline, e.g., clicking **Build Now**
 2. Jenkins tells Docker: “I need a container to run this pipeline”
 3. Docker creates a temporary container on-demand
 4. Once the pipeline finishes, the container is automatically deleted

3 On the Jenkins/EC2 instance:

- `docker ps` shows **no running containers**
- `docker ps -a` may show past containers (like the hello-world test)

4 Workflow with Docker Agent:

- Jenkins requests a container from Docker to execute the pipeline
- Docker provides a container (e.g., C1)
- Pipeline runs inside this container
- After completion, Docker terminates the container automatically

5 Key Advantages:

6 Containers exist only when needed

- Saves **cost** and **resources**
- Avoids **idle worker nodes**
- Provides a **clean, isolated environment** for every pipeline run

Multi-Stage, Multi-Agent Jenkins Pipelines

<https://github.com/iam-veeramalla/Jenkins-Zero-To-Hero/tree/main/multi-stage-multi-agent>

1 Concepts:

- Multi-stage: Pipeline divided into multiple stages (e.g., front-end, back-end, database).
- Multi-agent: Each stage can run on a different environment or Docker container.
- Useful for multi-tier architectures: front-end, back-end, database.

2 Example Architecture:

- Front-end: Node.js container.
- Back-end: Java with Maven container.
- Database: MySQL or Oracle container.
- Each container is created on-demand and terminated after the stage completes.

3 Pipeline Setup:

- Use `agent none` at the top level.
- Assign specific agents per stage:
 - `agent { docker { image 'node:14' } }` for front-end
 - `agent { docker { image 'maven:3.8-jdk-11' } }` for back-end
 - `agent { docker { image 'mysql:8' } }` for database
- Add steps inside each stage:
 - Front-end: `npm install, npm run build`
 - Back-end: `mvn clean install, mvn package`
 - Database: SQL scripts, migrations.

4 Execution:

- Multiple containers are created dynamically per stage.
- Containers are deleted after stage completion.
- Ensures isolated environments and reduces maintenance overhead.

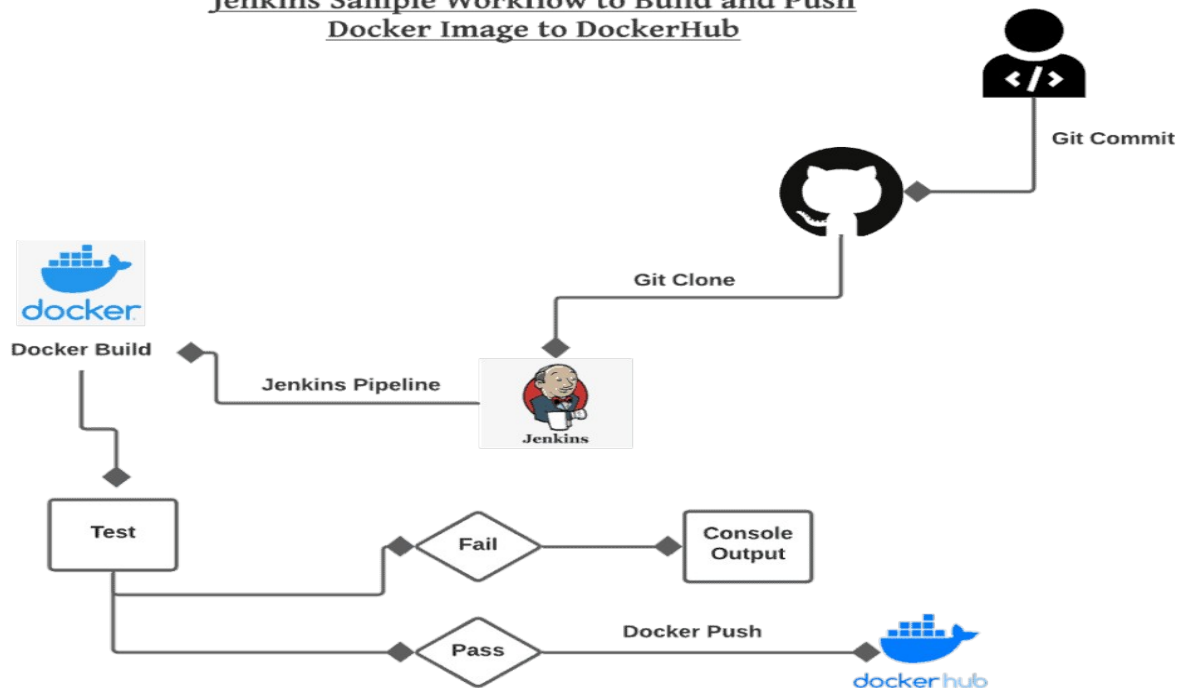
5 Advantages Over VM-Based Approach:

- No manual updates for software versions or dependencies.
- Saves cost and resources by spinning up containers only when required.
- Avoids maintaining multiple VMs for different technologies.
- Simplifies multi-tier pipeline execution.

6 Now using Jenkins Pipeline from SCM with Docker Agent

- **Open your first Jenkins job**
- Select “**Pipeline script from SCM**” instead of writing the pipeline manually
- Choose your source code repository (e.g., GitHub)
- Enter the repository URL
- Select the branch (Master/Main)
- Specify the path to the Jenkinsfile in the repository (e.g. [multi-stage-multi-agent](#)/Jenkinsfile)
- **Save the configuration**
- **build**
- [CI/CD \(Continuous Integration / Continuous Delivery\) Overview](#) as soon as u run it u can see a new docker pipeline running
- **docker ps** //u will see node related

Jenkins Sample Workflow to Build and Push Docker Image to DockerHub



AWS Console EC2 Setup Guide

1 AWS Console Login

- URL: <https://eu-north-1.console.aws.amazon.com/console/services?region=eu-north-1>
- Email: ad1768871@gmail.com
- Password: Jalaja@2320

2 Navigate to EC2

- On the AWS Console homepage, click **All Services**.
- Under **Compute**, select **EC2**.
- You are now on the **EC2 Dashboard**.

3 Launch Instance

1. Name: tutorial_instance
2. OS: Ubuntu (Free Tier eligible)
3. Instance type: t3.micro (Free Tier)

4. Create new key-pair:

- Name: `tutorial_key`
- Key pair type: RSA
- Format: `.pem`
- A file gets downloaded: `tutorial_key.pem`

4 Network Settings

- Allow SSH traffic ...anywhere

5 press Launch instance

- Successfully initiated launch of instance (`i-00abc93460b2e605f`)
- check here for all instances : <https://eu-north-1.console.aws.amazon.com/ec2/home?region=eu-north-1#Instances>:

6 now in terminal

- `cd /home/ee213082/Desktop/motorola`
- `sudo chmod 400 tutorial_key.pem`
- `ssh -i <location of pem file> ubuntu@<ip of instance>`
 - `ssh -i tutorial_key.pem ubuntu@13.60.209.31`
- now we are on our instance
 - `ubuntu@ip-172-31-27-24:~$`
 - `pwd // /home/ubuntu`

7 Update System Packages

- `sudo apt update -y`
- `sudo apt upgrade -y`

8 Install Java (required for Tomcat & Jenkins)

- Install OpenJDK 11: `sudo apt install openjdk-11-jdk -y`
- `sudo apt install openjdk-17-jdk -y`
- `sudo update-alternatives --config java`

- **Select 1 and press Enter.**
- Verify installation: **java -version**
openjdk version "11.0.16" 2025-07-15

9 Install Tomcat

- Move to /opt directory:
- Download Tomcat
`sudo wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.108/bin/apache-tomcat-9.0.108.tar.gz`
- Extract Tomcat:
`sudo tar xvf apache-tomcat-9.0.108.tar.gz`
- Rename folder for simplicity:
`sudo mv apache-tomcat-9.0.108 tomcat9`
- Make scripts executable and start Tomcat:
- `sudo -i` is a command in Linux that opens a **root shell** (a shell running as the root user).

```
sudo -i
```

```
root@ip-172-31-27-24:~#
```

```
cd /opt/tomcat9/bin
```

```
chmod +x *.sh //make scripts executable
```

```
./startup.sh //start tomcat
```

- Tomcat will start on **port 8080**.
- Open browser: `http://13.60.209.31:8080` → Tomcat welcome page should appear.
- If not!
- Go to **Security** → **Existing Security Groups** → **Edit inbound rules** → **Add rule**
 - Select **Custom TCP**
 - Port: **8080**
 - Source: **Anywhere (0.0.0.0/0)**
 - Inbound security group rules successfully modified on security group ([sg-0798026d03022268a | launch-wizard-1](#))
- Open browser: `http://13.60.209.31:8080` → Tomcat welcome page should appear.

10 Deploy Jenkins

- Move to Tomcat webapps folder:
`cd /opt/tomcat9/webapps`
- Download Jenkins WAR file:
`sudo wget https://get.jenkins.io/war-stable/latest/jenkins.war`
- `# ls -lh /opt/tomcat9/webapps/`

```
total 84M
drwxr-x--- 16 root root 4.0K Aug 18 11:07 docs
drwxr-x---  7 root root 4.0K Aug 18 11:07 examples
drwxr-x---  6 root root 4.0K Aug 18 11:07 host-manager
drwxr-x--- 10 root root 4.0K Aug 18 12:13 jenkins
-rw-r--r--  1 root root  84M Jul 23 11:36 jenkins.war
drwxr-x---  6 root root 4.0K Aug 18 11:07 manager
drwxr-x---  3 root root 4.0K Aug 18 11:07 ROOT
root@ip-172-31-27-24:~#
```

- After Tomcat is started, you can access Jenkins at:
`http://<your-ec2-public-ip>:8080/jenkins`
-

11 Get Jenkins Initial Admin Password

- `sudo cat /opt/tomcat9/.jenkins/secrets/initialAdminPassword`
- Copy this password into the Jenkins web page to unlock it.
- After this, **Jenkins is fully set up** and you can start creating jobs, linking GitHub, and running pipelines.

Tomcat

workflow

GitHub commit ---> Jenkins (watch repo) ---> Build .war ---> Deploy to Tomcat --> Access via browser

What is a .war file?

- .war stands for **Web Application Archive**.
- It's essentially a **ZIP file** that contains **all the code, libraries, and resources** for a Java web application.
- Contents usually include:

myapp.war

```
├─ WEB-INF/  
|   ├── web.xml    # deployment descriptor  
|   └─ classes/    # compiled Java classes  
|   └─ lib/        # external JAR libraries  
├─ index.jsp  
├─ css/  
└─ images/
```

Why .war?

- Java web servers like **Tomcat** understand .war files.
- When you deploy a .war file to Tomcat's webapps folder, Tomcat **automatically unpacks it** and serves the application.
- It's basically a **packaged Java web app ready to run**.

- Go to your **home directory**:

```
cd ~
```

- **Download Jenkins WAR:**

```
sudo wget  
https://get.jenkins.io/war-stable/latest/jenkins.war
```

- **Copy Jenkins WAR to EC2:**

```
scp -i /home/ee213082/Desktop/motorola/tutorial_key.pem  
/home/ee213082/jenkins.war ubuntu@13.60.209.31:/home/ubuntu/
```

- **SSH into EC2:**

```
ssh -i /home/ee213082/Desktop/motorola/tutorial_key.pem  
ubuntu@13.60.209.31
```

- If you are in `sudo -i`, type `exit` to go back to normal user

- **Check that Jenkins WAR is on EC2:** `ls -lh /home/ubuntu/jenkins.war`

- **Move Jenkins WAR to Tomcat webapps to deploy it:**

```
sudo cp /home/ubuntu/jenkins.war /opt/tomcat9/webapps/
```

Interview questions

1.what is jenkins,what is its primary use?

Jenkins is an open source automation server used for Continuous integration and continuous delivery(CI/CD)

Purpose:Jenkins is used to automate the process of building,testing,deploying of s/w applications, to allow teams to deliver the high quality softwares faster

2.explain difference b/w jenkins and hudson?

Hudson was used before jenkins as ci/cd tool

jenkins is a fork of hudson

bcz of some dispute over prjct governance,jenkins was created.

Since then k=jenkins is popular,open-src,continuous contributors

3.what is jenkins pipeline?

Jenkins pipeline is a set of plugins allows team to define software delivery process as code

creation of complex workflows like build,test,deployment stages ,which can be version controlled alongside application code

4.how do u define jenkins pipeline

Can be defined using jenkinsfile,which is text based file,written in groovy syntax

consist of stages,steps and configuration of pipeline..allowing for automated and repeatable s/w delivery process

5.explain diff b/w scripted and declarative jenkins pipeline

Jenkins pipeline=

- scripted pipeline
- declarative jenkins pipeline

scripted

- pipeline written in groovy syntax
- flexibility and power

more coding effort needed

declarative pipeline

- simpler
- structured syntax
- easier to read, write & maintain
- used more than scripted

Scripted Pipeline: It is based on Groovy script as their Domain Specific Language. One or more *node* blocks does the core work throughout the entire pipeline

Jenkinsfile (Scripted Pipeline)

```
node {  
  ①  
  stage('Build') { ②  
    // ③  
  }  
  stage('Test') { ④  
    // ⑤  
  }  
  stage('Deploy') { ⑥  
    // ⑦  
  }  
}
```

- ① Execute this pipeline or any of its stages on any available agent
- ② Defines the Build stage
- ③ Performs steps related to Build stage
- ④ Defines the Test stage
- ⑤ Performs steps related to Test stage
- ⑥ Defines the Deploy stage
- ⑦ Performs steps related to Deploy stage

Declarative Pipeline: It provides a simple and friendly syntax to define a pipeline. Here, *pipeline* block defines the work done throughout the pipeline

Jenkinsfile (Declarative Pipeline)

```
pipeline {  
  agent any ①  
  stages {  
    stage('Build') { ②  
      steps {  
        // ③  
      }  
    }  
    stage('Test') { ④  
      steps {  
        // ⑤  
      }  
    }  
    stage('Deploy') { ⑥  
      steps {  
        // ⑦  
      }  
    }  
  }  
}
```

- ① Execute this pipeline or any of its stages on any available agent
- ② Defines the Build stage
- ③ Performs steps related to Build stage
- ④ Defines the Test stage
- ⑤ Performs steps related to Test stage
- ⑥ Defines the Deploy stage
- ⑦ Performs steps related to Deploy stage

6.what is jenkins agent(node) & how does it work?

- Whenever we setup up jenkins,we have a jenkins server running(where jenkins is running).And then we can have lots of remote machine that can be connected to Jenkins server..so these remote machine is called jenkins node..they can phy/virtual..they can execute tasks(like 1 server for building,1 for testing procee,,another for deployment).Generally we dont put load on jenkins server,so we make use of these machines and give jobs
- Agents can run on same machine as the jenkins server(master) or on remote machine
- these node/agent communicate with jenkins to rcv build instr and report results
- Jenkins agent also known as Node,is a machine(physical/virtual) that executes tasks as apart of jenkins build process

7.explain purpose of jenkins plugins?

Jenkins plugins extend the functionality of the jenkins automation server by providing additional features ,integration & capabilities

plugins can be used to support various tasks such as src code managemnt,build tools,testing f/w,deployment platforms & notification system;

8.How do u secure jenkins

- Only authenticated users can access jenkins
- enable authentication mechanisms like;LDAP,active directory,oAuth
- Access control can be configured using **role -based security** which defines permission for users and grps
- jenkins connections can be **encrypted using https**
- **security plugins**

9.Explain the concept of jenkins job DSL?

- Jenkins (DSL) Domain specific language is a plugin that allows user to define jenkins job programmatically using a groovy based DSL
- it enables creation of job,views,other configuration of pipelines,which can be version-controlled alongside application code

10.what is jenkins file,how is it used

Jenkinsfile is txt file written in groovy syntax,that defines configuration of jenkins pipeline

- has stages,steps,and post -build actions
- s/w delivery process as code
- can be version controlled alongside app code

11.Explain Jenkins build triggers and how they work

A build trigger is a condition or event that automatically start a build

Trigger can occur due to events like;

- scm commit
- time schedules(using cron syntax)
- the completion of other builds
- manually

12.How do u automate deployment using jenkins

For automated deployment use

- jenkins pipeline or
- freestyle prjcts
- Jenkins can execute deployment scripts
- interact with deployment tools(kubernetes,dockes,aws)
- trigger deployment based on pre-defined condition or user inputs

13.jenkins build artifacts?how r they used?

- build artifacts are files generated during build process such as compiled binaries,test results,documenation & archives
- Jenkins artifacts r stored on jenkins server or ext artifact repo like:jfrog,nexus..then we can access or use them

14.jenkinspipeline stges

- It has stages= distinct phases /steps
- stages;build,test,deploy,notify
- each stages has one or more steps or actions
 - can be configured to run sequentially,,run one after another
 - or paralley
 - conditinally based on criteria

15. troubleshoot jenkins build failure?

- Build logs
- console o/p
- error msg
- enable verbose loggings
- jenkins server and agent configs
- code changes,dependenices,,check

16. what are some of the default environment variables in jenkins

- \$JOB_NAME
- \$NODE_NAME
- \$WORKSPACE
- \$BUILD_URL
- \$JENKINS_URL

17.WHY DO U USE PIPELINE IN JENKINS?

- It enables us to write or pipeline code in version-controlled repo along with our src coe
- it supports parallel execution,error handling,shared libraries

18 what happens when jenkins agent is offline?and what is the best practise in that situation?

- When a Jenkins agent is offline, any job tied specifically to that agent (or its node) will remain in the **build queue**. The job won't run until the agent comes back online or becomes free. In some cases, this can cause jobs to **wait indefinitely** if Jenkins doesn't know the node is unavailable.

Best Practice:

Instead of binding jobs to a single agent, it's better to use **labels**. Labels group multiple agents with similar environments. When a job is tied to a label, **any available agent under that label** can pick up the job. This ensures higher availability, reduces build waiting time, and avoids a single point of failure if one agent goes offline.

Additionally, organizations often set up:

- **Monitoring/alerts** for agent disconnects
- **Auto-reconnect mechanisms** (systemd, cloud provisioning, Kubernetes, etc.)
- **Redundant agents** with the same label for critical pipelines

19.What are the scopes of Jenkins credentials?

Jenkins credentials can have **two scopes**:

1. **Global Scope**

- Credentials are available to **all jobs, pipelines, and projects** within the Jenkins instance.
- Typically used when jobs need to authenticate with external systems (e.g., GitHub, Docker registry, cloud services).
- Example: A GitHub access token used by multiple jobs for SCM checkout.

2. System Scope

- Credentials are **only available to Jenkins itself**, not to user jobs.
- Used internally when Jenkins as a service needs to authenticate with external systems or infrastructure.
- Example: Jenkins authenticating with an LDAP server, or sending email through an SMTP server.

19. How do you store credentials in Jenkins securely?

Since credentials are sensitive (passwords, SSH keys, tokens, etc.), Jenkins provides multiple secure mechanisms:

1. Credentials Plugin (built-in)

- Main way to store and manage secrets inside Jenkins.
- Supports username/password, SSH private keys, API tokens, certificates, etc.
- Credentials can be stored with **global** or **project/folder-specific scope**.
- With the **Credentials Binding Plugin**, these secrets can be injected as environment variables into jobs or pipelines (without hardcoding them in scripts).

2. Secret Text Credentials

- An extension of the credentials plugin.
- Allows storing secrets as plain strings (API tokens, keys, single secret values).
- These can also be injected into jobs/pipelines securely.

3. HashiCorp Vault Plugin (external secret manager integration)

- Jenkins can be integrated with **Vault** for enterprise-grade security.
- Secrets are not stored in Jenkins directly; instead, Jenkins fetches them dynamically from Vault at runtime.
- This ensures secrets are always fresh, centrally managed, and audited.

20. how can you find log files in Jenkins?

- Using web interface: you can access log files by going to manage Jenkins -> system log -> or /log on your Jenkins URL

- using file system: u can access the log files by going to the JENKINS_HOME diretcory on your jenkins server.

21.how do u integrate static code analysis into jenkins pipeline?

- **Static code analysis** tools (like **SonarQube**, **Checkstyle**, **PMD**, or **FindBugs/SpotBugs**) help automatically scan source code for errors, code smells, and security vulnerabilities.
- Steps to integrate into Jenkins Pipeline:
 1. **Install the required plugin**
 - From *Manage Jenkins* → *Manage Plugins*, install tools like the **SonarQube Scanner plugin**, **Checkstyle plugin**, or **Warnings NG plugin**.
 2. **Configure the tool in Jenkins**
 - Go to *Manage Jenkins* → *Global Tool Configuration* and set up the path to the scanner/CLI.
 - For SonarQube, also configure the **SonarQube server details** and authentication token.
 3. **Add pipeline steps**
 - In the Jenkinsfile (pipeline script), add a stage to run static code analysis during the build process.
 - Example (SonarQube):


```
stage('Static Code Analysis') {
    steps {
        withSonarQubeEnv('MySonarServer') {
            sh 'mvn sonar:sonar'
        }
    }
}
```
 4. **Publish and review results**
 - Configure the pipeline to collect analysis results.
 - Use the plugin to publish reports (SonarQube Quality Gate, Checkstyle reports, etc.) directly in the Jenkins UI.
 - Optionally, configure Jenkins to **fail the build** if violations exceed a threshold.

“To integrate static code analysis into a Jenkins pipeline, I first install the relevant plugin such as SonarQube Scanner or Checkstyle. Then I configure the tool under Jenkins Global Tool Configuration. In the pipeline (via Jenkinsfile), I add a dedicated stage that executes the analysis command — for example, `mvn sonar:sonar` for SonarQube. Finally, I publish the results back into Jenkins, where developers can review reports, and I can even enforce quality gates so that

builds fail if there are critical issues. This way, static code analysis becomes an automated part of the CI/CD workflow.”

22. explain how can u move or copy jenkins from 1 server to another server?

1. Moving Entire Jenkins Server

If you want to **move Jenkins completely** from one server to another:

- **Stop Jenkins** on the old server.
- **Copy the \$JENKINS_HOME directory** (this contains all jobs, plugins, config.xml, users, secrets).
- **Install Jenkins** on the new server.
- **Paste \$JENKINS_HOME** into the new server's Jenkins home.
- **Start Jenkins.**

This way, everything (jobs, users, plugins, configs) moves to the new server.

2. Copying Individual Jenkins Jobs

If you only want **specific jobs**:

- Each job has its own directory under:
\$JENKINS_HOME/jobs/<job-name>/
 - You can **copy that directory** to another server inside the same location (jobs/).
 - Restart Jenkins or click “**Reload Configuration from Disk**” in Manage Jenkins.
-

3. Cloning a Job in Same Jenkins

If you just want to **duplicate a job inside same Jenkins**:

- Option A: Use Jenkins UI → "New Item" → Copy from existing job.
- Option B: Copy the job directory and **rename the folder**, also update the `config.xml` inside.

23. how can u turn off jenkins security if the adminstrative users have locked themselves out of the admin console?

- JENKINS_HOME folder contains a file named config.xml ,when u enable the security this file contains an xml element named useSecurity that changes to true
- change it to:
`<useSecurity>false</useSecurity>`
- restart jenkins

24. how can we implement a rolling update deployment strategy using jenkins?

- Create a pipeline that builds and tests your application
- using a deployment tool like kubernetes or a cloud platform's deployment service
- configuring the deployment to update instances gradually
- monitoring the deployment and implementing rollback mechanisms

25. use of JENKINS_HOME directory?

- Where all settings, logs, configuration are stored
- stores all these info in xml files

26. is jenkins enough for automation?

- No, jenkins is not enough for automation
- jenkins is an automation server used for CI/CD process
- we need tools like git, maven, docker, kubernetes to work with jenkins, so it works as an orchestrator for automation

27. what is continuous testing

- Continuous testing is the process where you execute automated tests as a part of software delivery pipeline.
- This is done so that you get the feedback on the business risk associated with the software as early as possible.

28. What is Kubernetes? and how can you integrate Jenkins with Kubernetes?

Kubernetes is an open-source platform used to **automate the deployment, scaling, and management of containerized applications.**

- Reporting the issue immediately to stakeholders with all relevant details,
- collaborating with the development team to assess the impact and find a quick resolution,
- testing the fix thoroughly to ensure no additional issues are introduced.

29. How to Create a Continuous Deployment in Jenkins?

1. Create a pipeline job.
2. Configure the build triggers.
3. Configure the deployment stage.
4. Configure the post-build actions.

30. Explain your current ci/cd setup

- there are many ci tools, i will consider Jenkins to answer this
- CI/CD is a process of continuous integration & continuous delivery
- in the current org, we used GitHub as source code repository
- we have configured a GitHub webhook that triggers a Jenkins pipeline on every commit
- the Jenkins pipeline is setup with multiple stages
 - build stage : take care of static code analysis, unit testing & build
 - test stage - running smoke and functional tests
 - deploy stage – push artifacts to the registry, prepare deployment manifest and deploy on cluster

31. Explain master-slave architecture of Jenkins?

- Jenkins master pulls the code from the remote GitHub repository every time there is a code commit.
- it distributes the workload to all the Jenkins slaves.
- On request from Jenkins master, the slave carries out build and test and produces test reports.

32.which cmd runs jenkins from the cmd line?

Java -jar jenkins.war

33.which file is used to define dependency in maven?

- Pom.xml

34.mention cmds to restart jenkins

- jenkins url /restart =force restrat without waiting build to complete
- jenkins url /safeRestart = wait fro build to complete

35.What is the difference between a Freestyle Job and a Pipeline Job in Jenkins?

- **Freestyle Job:**
 - A simple, UI-based job configuration.
 - You define build steps (e.g., Execute Shell, Windows Batch, Invoke Ant, Maven goals) using the Jenkins web interface.
 - Best suited for small or straightforward tasks like compiling, running tests, or triggering a script.
 - Limited flexibility — harder to maintain for complex workflows.
- **Pipeline Job:**
 - Uses a **Jenkinsfile** (written in Groovy syntax) to define the build, test, and deploy pipeline as code.
 - Supports multiple **stages** and **steps**, conditional execution, loops, and parallel execution.
 - Pipeline as Code → can be version-controlled in Git.
 - Two types:
 - **Declarative Pipeline** → structured, easy to read/write, most commonly used.
 - **Scripted Pipeline** → more flexible and powerful, but requires more Groovy knowledge.

Main Difference:

- **Freestyle Job** = Configured via UI, good for simple builds.

- **Pipeline Job** = Code-based (Jenkinsfile), supports complex CI/CD workflows, scalable and maintainable.
-

That's the neat, interview-ready an

When you open Jenkins and press “**New Item**”:

You will see options like:

- **Freestyle Project**
 - **Pipeline**
 - (and some others like Multibranch Pipeline, Maven project, etc.)
-

If you choose Freestyle Project:

- Jenkins opens a **form in the UI**.
- You add things like:
 - **Source Code Management** → Git repo URL
 - **Build Steps** → e.g., "Execute Shell", "Invoke Ant", "Maven goals"
 - **Post-build Actions** → e.g., "Send email", "Archive artifacts"
- Everything is **configured by clicking and filling boxes**.
- Example: If you want to compile code, you just add a step "Execute Shell" → `gcc main.c -o main`.

So, Freestyle = **Click + Fill in UI forms**.

If you choose Pipeline:

- Jenkins asks you to either:
 1. Write pipeline script directly in Jenkins (Groovy code).
 2. Or point to a **Jenkinsfile** inside your Git repo.

- A **Jenkinsfile** looks like this:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'make'
      }
    }
    stage('Test') {
```

```

    steps {
        sh 'make test'
    }
}
}

```

- This defines steps **as code** → stages (Build, Test, Deploy).
- The Jenkinsfile can be pushed to Git → version controlled.

So, Pipeline = **Code-based automation, not just clicking UI.**

Easiest Way to Remember:

- **Freestyle Job** → You do everything by **clicking in Jenkins UI.**
- **Pipeline Job** → You write a **Jenkinsfile (Groovy script)** with stages and steps.

36. When you launch an EC2 instance on AWS, what are the key things you need to configure before it starts running?

When launching an EC2 instance, we configure:

- Select **AMI** (Amazon Machine Image) -ubuntu, windows, linux
- Select **Instance Type**-e.g., t2.micro, t3.medium.
- Configure **Key Pair** for SSH
- Configure **Network/VPC & Subnet**
- Set **Security Group** inbound/outbound rules
- Configure **Storage (EBS volume)**
- (Optional: IAM role, tags, monitoring options)

37. You have a Jenkins pipeline that builds a .jar file and you need to deploy it to an AWS EC2 instance. How do you securely connect Jenkins to the EC2 instance and deploy the application?

Secure Jenkins → EC2 Deployment

1. Use SSH Key Pair

- Add the EC2 instance key pair (.pem) to **Jenkins credentials** for secure access.

2. Configure Security Group

- Allow SSH (port 22) from Jenkins server's IP.

3. Connect via Pipeline

- Use scp to copy the .jar file to EC2.
- Use ssh to run deployment commands (java -jar ...).

4. Automation

- Integrate these steps in the **Jenkinsfile** pipeline for fully automated deployment.

38.What is a GitHub webhook in Jenkins? How does it help in CI/CD?

- **GitHub Webhook:** A webhook is a trigger from GitHub to Jenkins.
- When a commit or push happens in the GitHub repository, the webhook **sends a payload to Jenkins**, notifying it that changes occurred.
- Jenkins can then **automatically start the build job**, enabling **continuous integration** without manual intervention.

39.cron syntax

In **Jenkins**, the **cron syntax** is used when you configure **Build Periodically** or **Poll SCM**. It looks like Linux **cr**on, but there are **important differences**.

MINUTE HOUR DOM MONTH DOW

0 5 1 8 2

- Minute = 0 → At minute 00
- Hour = 5 → At 05:00 AM
- Day of Month = 1 → On the 1st day of the month
- Month = 8 → In August
- Day of Week = 2 → On Tuesday (0/7 = Sunday, 1 = Monday, 2 = Tuesday ...)

Jenkins Cron Syntax Format

It has **5 fields**:

MINUTE HOUR DOM MONTH DOW

- **MINUTE** → 0–59
- **HOUR** → 0–23

- **DOM (Day of Month)** → 1–31
- **MONTH** → 1–12 (or names: JAN, FEB, ...)
- **DOW (Day of Week)** → 0–7 (where 0 and 7 = Sunday, or names: MON, TUE, ...)

Special: Jenkins adds an optional H for “hash” (to spread load).

Special Characters

1. * → Wildcard

- Means “**any value**” in that field.
- Example:

* * * * *

→ Runs every **minute, every hour, every day, every month, every weekday**.

30 * * * *

→ Runs every hour at the 30th minute. (Because * in HOUR means "any hour").

2. H → Hash (Jenkins-specific)

- **Unique to Jenkins** (not in Linux cron).
- Jenkins calculates a **stable pseudo-random value** based on the job name.
- Purpose: **spread out job execution** so that not all jobs run at the same time (e.g., midnight).
- Example:

H * * * *

→ Runs once per hour, at a **specific minute chosen by Jenkins** (different for each job).

H H * * *

→ Runs once per day, at a **random but stable hour+minute**.

This avoids server overload when many jobs are scheduled.

• 3. */n → Every n units

- Means “**every nth value**” in that field.

- Example:

`* / 5 * * * *`

→ Every **5 minutes** (00:00, 00:05, 00:10 ...).

`0 * / 2 * * *`

→ Every **2 hours** (02:00, 04:00, 06:00 ...).

`15 9-17 / 2 * * *`

→ “Run at the **15th minute of every 2nd hour between 9 AM and 5 PM, every day.**”(so 9:15, 11:15, 13:15, 15:15, 17:15).

- **Minute = 15**

→ Always runs at the 15th minute.

Example: 09:15, 11:15, etc.

1. **Hour = 9-17/2**

- 9-17 → This is a range → from 9 AM to 5 PM (17 = 5 PM).

- /2 → Step value → take **every 2nd hour** in that range.

- So it picks: **9, 11, 13, 15, 17.**

2. **Day of Month = * (any)**

→ Runs every day.

3. **Month = * (any)**

→ Runs every month.

4. **Day of Week = * (any)**

→ Runs regardless of weekday.

- A-B → Range (e.g., 1-5 = Monday to Friday).

- A, B, C → List (e.g., MON, WED, FRI).

Jenkins vs Linux Cron

- **Linux cron** runs *exactly* at the specified time.
- **Jenkins cron** adds H to avoid job overload on the master node.
- Example:

- Linux: `0 0 * * *` → always midnight.

- Jenkins: `H H * * *` → sometime every day, but distributed across nodes.

Cicd interview question

1 what is CI/cd?

- Continuous integration and continuous delivery/deployment
- it is a set of practices used by development and operations team for s/w development and deployment

2 continuous integration v/s continuous delivery v/s continuous deployment?

- Continuous integration is a development practice where developers frequently integrate their code changes into a shared repository.
- In continuous delivery, all code changes in the branch like feature addition or bug fixes or configuration change are automatically deployed to the testing or production.
- In continuous deployment, every change that passes the automated test is deployed to the production automatically.
- **CI:** Automatically build and test code on every commit using Jenkins.
- **Continuous Delivery:** Prepare artifacts for deployment and store in artifact repository; deployment may be manual.
- **Continuous Deployment:** Automatically deploy artifacts to production without manual intervention.

3 Benefit of cicd pipeline?

Benefits of CICD pipeline

- higher efficiency,
- reduced risk of defect
- , faster delivery,
- reduced deployment risk,
- better collaboration,
- automated process.

4 . Explain different stages in a CI-CD pipeline.

- Code Commit,

- Build,
- Test,
- Deploy,
- Release,
- Monitor, and Feedback

5 popular CICD tools?

- Jenkins,
- CircleCI,
- Bamboo,
- TeamCity,
- TravisCI,
- AzureDevOps

6 What are the different tools you have used while implementing CI-CD pipeline?

- Version control: Git(GitHub, GitLab, Bitbucket.)
- CI-CD platform : Jenkins, CircleCI, Bamboo, TeamCity, TravisCI, Azure DevOps.
- Testing:JUnit, TestNG for Java, PyTest for Python, Mocha for JavaScript.
- Deployment:Docker for containerization, Kubernetes for orchestration, Ansible for configuration management
- Monitoring:Prometheus, Grafana.

7 How do you trigger a CI-CD pipeline?

- Code Push,
- Scheduled Trigger,
- Manual Initiation,
- Continuous Integration.

8 How to write a CI-CD pipeline in Jenkins?

- install Jenkins and required plugins.
- Set up version control such as Git.
- Create a Jenkins file with build, test, and deployment stages, and save Jenkins file to the repository.
- Then, create a Jenkins job, link to the Jenkins file, and trigger the job for the automated CI-CD process.

9 What r some best practice for securing a cicd pipeline?

- Limiting access to the CICD system,
- ensuring secure storage of secrets and credentials,
- regularly updating and patching CICD tools,
- implementing code scanning and vulnerability assessments,
- using signed artifacts and checksums for deployment.

10 Challenges of CI/CD Implementation

1. Tool Integration

- Different teams use different tools (Git, Jenkins, Docker, Kubernetes). Getting them to work together smoothly is tricky.

2. Legacy Systems

- Older applications may not support automated builds, testing, or deployments. Migrating them is challenging.

3. Test Automation Gaps

- CI/CD relies on fast, reliable tests. If tests are missing or flaky, pipelines break often.

4. Security & Compliance

- Automating deployments might introduce security risks if secrets, credentials, or compliance checks aren't handled properly.

5. Infrastructure Complexity

- Scaling pipelines across multiple environments (dev, QA, prod, cloud) requires strong infrastructure and monitoring.

6. Cultural Resistance

- Teams used to manual processes may resist automation, fearing job changes or added complexity.

7. Pipeline Performance

- Long build/test times slow down feedback, defeating the purpose of CI/CD.
Optimizing performance is hard.
-